

Sommario

1. Architetture superscalari.....	1
1.1 Hardware di gestione dei problemi tra le pipe	6
1.2 Esempio	7
1.3 Salti nelle architetture superscalari	8
2. Memoria	9
2.1 Trattazione senza memoria virtuale.....	10
2.2 Tecniche di gestione	11
2.2.1 Memoria Totalmente Associativa (Una chiave = Un elemento):	12
2.2.2 Organizzazione a Blocchi (Divisione dell'indirizzo):	12

1. Architetture superscalari

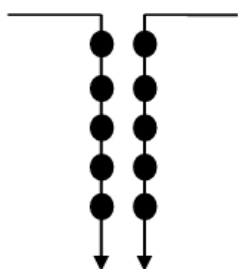
Creiamo la pipe non per avere un tempo minore di elaborazione della singola istruzione, ma perché vogliamo migliorare la produttività. Tempo di attraversamento della singola istruzione è lo stesso, ma il tempo di produttività migliore. Per avere prestazioni ancora migliori posso pensare di avere un'architettura che presenti più pipe che eseguono diverse istruzioni in parallelo.

Queste architetture con più pipe sono chiamate **superscalari**.

Realizzare queste architetture introduce complessità e alcune problematiche:

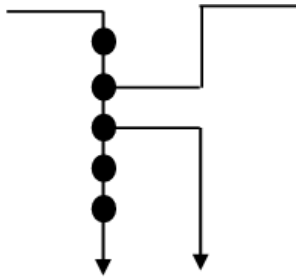
- 1) Le pipe devono condividere un unico accesso alla memoria comune per il prelievo delle istruzioni. Questo significa che due pipe diverse non possono avviare l'esecuzione di due istruzioni nello stesso istante in quanto non possono essere prelevate contemporaneamente dalla memoria, ma in modo sequenziale.

Se capita il caso in cui le pipe sono totalmente disgiunte, cioè non condividono alcuna unità funzionale, non si riscontrano ulteriori problemi, a parte i problemi interni della singola pipe come ad esempio la branch prediction.



le linee nere con le frecce sono il percorso logico che compie un'istruzione. I pallini rappresentano le risorse hardware ovvero i vari stadi o unità funzionali della pipe. Sarebbe la soluzione ideale. Non ci saranno mai conflitti perché ogni pipe ha le sue risorse dedicate. Lo svantaggio è che ogni singola pipe avendo dei componenti dedicati farebbe occupare più silicio e maggiore consumo energetico.

Figura 23: Due pipe disgiunte.



qui vediamo che alcuni stadi sono in comune quindi i due flussi useranno la stessa unità funzionale numero 2 e numero 3. Il problema è che si crea un collo di bottiglia perché due istruzioni devono letteralmente usare lo stesso circuito nello stesso momento il che è fisicamente impossibile. Il vantaggio è che uso meno unità funzionali perché sono condivise.

Figura 24: Due pipe non fisiologicamente giunte.

Il processore Pentium è stato uno dei primi esempi di processore con più pipe. Aveva due pipe le quali però non erano usate in modo efficiente perché una era sempre utilizzata, mentre l'altra entrava in gioco solo se le istruzioni sicuramente non creavano conflitti con le istruzioni in esecuzione nella pipe principale.

Quindi già da prima dell'esecuzione dovevo sapere quali erano le istruzioni e gli operandi che non potevano creare collisioni. Vorremmo quindi avere dei sistemi nei quali possiamo avere delle regole meno forti che ci consentano di aumentare l'efficienza di utilizzo della pipe. Vogliamo quindi capire i casi in cui è possibile usare contemporaneamente diverse pipe con punti in comune senza che creino conflitti nell'utilizzo delle unità fondamentali.

Ricordiamoci che il prelievo dell'istruzione è unico e quindi non è possibile prelevare 2 istruzioni contemporaneamente dalla memoria. Ciò significa che non è possibile avere un ingresso contemporaneo di 2 istruzioni sulle pipe, ma saranno sempre l'una sfalsata rispetto all'altra. Quindi non c'è una vera e propria alimentazione in parallelo.

A monte della fase di EX di ciascuna pipe possiamo inserire un'unità hardware detta **Dispatch** che analizza più istruzioni per volta e decide quali inserire nella pipe.

Esempio: per eseguire una moltiplicazione occorrono 7 stadi (o quanti di tempo) e per un'addizione ne servono 9. Immaginiamo la sequenza ADD1 e ADD2:

- **Ciclo 1:** Prelevo ADD 1.
- **Ciclo 2:** Prelevo ADD 2 (ADD 1 avanza verso la decodifica).
- **Ciclo 3:** Decodifica completata, il Dispatch analizza.
- **Ciclo 4: ADD 1 inizia l'esecuzione.** L'esecuzione durerà 9 cicli.
- **Ciclo 5: ADD 2 inizia l'esecuzione** in un'unità funzionale parallela. ADD 1 è al suo secondo ciclo di esecuzione.

Il Parallelismo: Dal Ciclo 5 fino al Ciclo 12, **entrambe le addizioni stanno fisicamente calcolando nello stesso momento**, occupando contemporaneamente i "muscoli" (le unità funzionali) del processore.

Se non avessi un'architettura con più unità specializzate (superscalare), la ADD 2 non potrebbe entrare in esecuzione al Ciclo 5; dovrebbe aspettare il Ciclo 13, cioè che la ADD 1 abbia finito tutti i suoi 9 stadi!

La fase di Dispatch non può essere software perché deve essere molto veloce ed efficiente, quindi realizzata in hardware.

$1,5 \times 10^3$ con $2,0 \times 10^1$

Se ho una moltiplicazione dei due numeri floating points dovrei

- 1) Sommare gli esponenti ($3+1$)
- 2) Moltiplicare le mantisse ($1,5 \times 2,0$)
- 3) Normalizzare ovvero se prima della virgola c'è solo un numero diverso da zero
- 4) Arrotondamento

Risultato finale: 3.0×10^4

Se guardiamo la somma invece avrei:

- 1) Sottrazione esponenti: bisogna capire prima chi è il più grande ($3-1=2$) → ora l'hw sa che c'è un disallineamento di due posizioni
- 2) Shift A/B: ora bisogna allineare le virgole. Da 2.0×10^1 diventa 0.02×10^3
- 3) Somma mantisse ($1,5+0.02$)
- 4) Normalizzazione
- 5) Arrotondamento

Paradossalmente è più complesso fare una somma che una moltiplicazione in FP. Si capisce comunque che alcune fasi sono comuni alle due operazioni di moltiplicazione e addizione. Quando occorre eseguire contemporaneamente un'addizione e una moltiplicazione in virgola mobile, sarebbe preferibile che entrambi i dispositivi presentassero il risultato contemporaneamente.

Se il sistema venisse alimentato contemporaneamente con un'addizione e una moltiplicazione non si può escludere che in qualunque delle stazioni comuni ci possa essere una collisione, specie se alcune unità vengono utilizzate in ciclo.

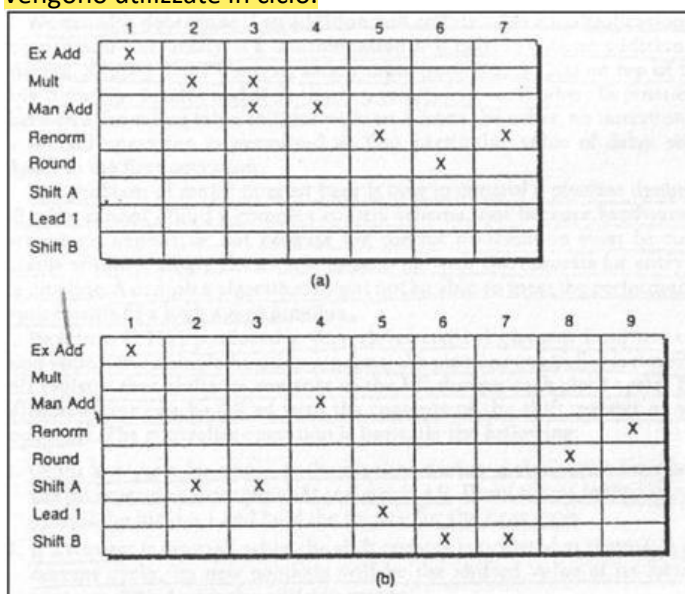


Figura 27: Moltiplicazione (a) e Addizione (b)

In questo schema possiamo vedere i passi della moltiplicazione e addizione utilizzando un sistema comune.

Per una moltiplicazione occorrono 7 stadi e per un'addizione occorrono 9 stadi. Alcuni stadi vengono percorsi più volte ad esempio la moltiplicazione fa due volte Man ADD di seguito poi ad esempio alcuni stadi non sono proprio usati dalla moltiplicazione e solo dall'addizione.

Se partissero in istanti sbagliati potrebbero quindi generare conflitti.

	1	2	3	4	5	6	7
Ex Add	X	Y					
Mult		X	XY	Y			
Man Add			X	XY	Y		
Renorm					X	Y	X
Round						X	Y
Shift A							
Lead 1							
Shift B							

(a)

Mettiamo caso di avere due moltiplicazioni di seguito (moltiplicazione X e moltiplicazione Y): sicuramente sono sfasate di uno perché prelevate in modo sequenziale. Quando la seconda moltiplicazione arriva alla fase EX vediamo che c'è collisione nelle celle XY.

Comprendiamo quindi che non è possibile attivare una moltiplicazione subito dopo un'altra; occorre aspettare un tempo opportuno. In altre parole, se si presentano due moltiplicazioni in simultanea l'hardware deve mandare avanti una delle due e bloccare l'altra per il tempo necessario a evitare collisioni.

Se invece avessimo un'addizione seguita da una moltiplicazione avremmo:

ex add	A	M							
mult			M	M					
man add			AM	M					
renorm						M		M	A
round							M	A	
shift a		A	A						
lead1					A				
shift b						A	A		

troviamo una collisione in man Add

Quindi una moltiplicazione non può partire subito dopo un'addizione. Questo concetto viene rappresentando ponendo un 1 come prima componente di un vettore binario detto **vettore delle collisioni**

Quindi dobbiamo capire quando far partire la moltiplicazione dopo l'addizione. Spostiamo la griglia della moltiplicazione a destra. Cioè sappiamo che non può partire un quanto di tempo dopo (momento di arrivo in EX), vediamo se può due quanti dopo

A • M	1	2	3	4	5	6	7	8	9
<i>ex add</i>	A		M						
<i>mult</i>				M	M				
<i>man add</i>				A	M	M			
<i>renorm</i>							M		AM
<i>round</i>								AM	
<i>shift a</i>		A	A						
<i>lead1</i>					A				
<i>shift b</i>						A	A		

Non può partire neanche due quant i dopo. Addirittura abbiamo due collisioni. Il vettore delle collisioni mette 1 in seconda posizione.

Dopo 3 quanti di tempo finalmente non c'è nessuna collisione quindi mettiamo 0. Si continua così fino a riempire il vettore delle collisioni.

1101000

Il vettore delle collisioni non può in alcun modo dirci la posizione della collisione, ma ci dice solo che se parto nell'istante i e la cella i ha un 1 allora avrò una collisione. L'informazione contenuta nel vettore non è ho una collisione in questa posizione o in questa, ma semplicemente che se fai partire l'istruzione in quella posizione allora avrai una collisione. Una cosa importante da sottolineare è che l'informazione di questo vettore può essere utile al compilatore perché se sa che devo aspettare 2 cicli per far partire la seconda istruzione, invece di far aspettare il processore a vuoto per due cicli, prenderà dal codice due istruzioni indipendenti e le incasterà in mezzo perappare i buchi di inefficienza.

È da notare che **non vale la proprietà commutativa**: mettendo prima la moltiplicazione e poi l'addizione, non si verificano mai conflitti quindi scambiando le istruzioni non ci assicura che si comporta allo stesso modo, ma dobbiamo ricalcolare il vettore dei conflitti. Questa cosa però è utile perché magari scambiando le istruzioni possiamo anche diminuire i conflitti o addirittura azzerarli.

Abbiamo definito quindi la sequenza **addizione-moltiplicazione**. Dobbiamo definire tutte le possibili combinazioni:

ADD ADD addizione seguita da un'addizione

ADD MUL moltiplicazione seguita da un'addizione

MUL MUL moltiplicazione seguita da una moltiplicazione.

Quindi se consideriamo una generica op_1 e op_2 dobbiamo considerare i casi in cui op_1 segue un'altra op_1 e op_1 segue una op_2 e anche viceversa quindi op_2 segue op_2 e op_2 segue op_1

Inoltre noi abbiamo considerato una sequenza di due operazioni, ma gli stadi della singola operazione sono più di due quindi potrebbero arrivare più operazioni e avere una sequenza più lunga e mischiata

Add mol mol add add mol...

In questa situazione abbiamo bisogno del vettore di collisione addizione seguita da moltiplicazione e moltiplicazione seguita da moltiplicazione, ma facendo così stiamo separando le istruzioni due a due, invece sono una di seguito all'altra quindi la terza operazione cioè la moltiplicazione deve tenere conto della moltiplicazione precedente e anche dell'addizione ancora precedente.

Il sistema di decisione in base alle collisioni deve essere tale che ogni istruzione non deve collidere con le istruzioni presenti nella pipe e quindi valutare eventuali collisioni anche con le istruzioni non direttamente precedenti.

Quindi a maggior ragione il compilatore deve avere visione dei vettori di collisione così da poter al meglio sfruttare le proprietà commutative e associative.

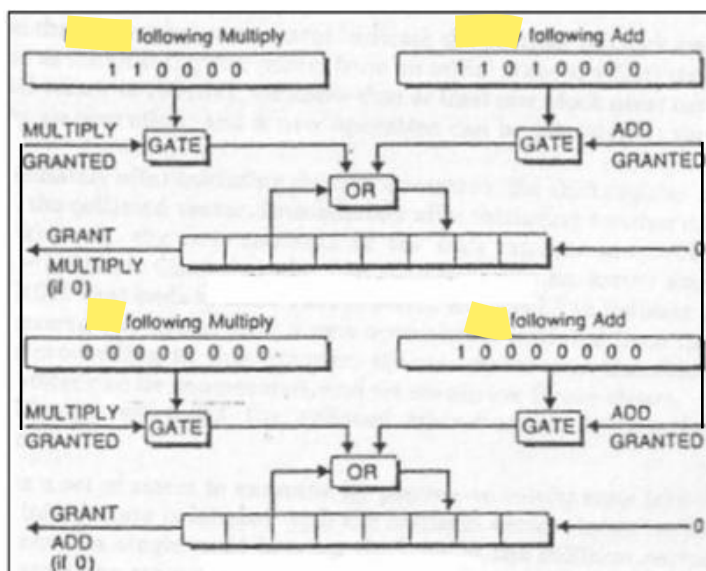
1.1 Hardware di gestione dei problemi tra le pipe

Definiamo i seguenti segnali:

- Multiply Grant abilita l'esecuzione di una moltiplicazione quando è 0
- Add Grant abilita l'esecuzione di un'addizione quando è 0
- Multiply granted indica che è stata eseguita una moltiplicazione, ovvero è entrata nella PIPE
- Add granted indica che è stata eseguita un'addizione.

l'hardware necessario per evitare le collisioni include:

- Registri a scorrimento (inizialmente settati a zero)
- Vettori delle collisioni
- Porte OR



I vettori delle collisioni sono statici perché sono calcolati una sola volta all'inizio. La dinamicità è fornita dallo shift register.

I registri a scorrimento sono due in modo che ce ne sia uno per concedere la moltiplicazione e uno per concedere l'addizione, ovvero il primo e se ho la mul come seconda istruzione e la devo far partire quindi devo capire quando concederla e l'altro per l'addizione come seconda istruzione.

Ad ogni colpo di clock fanno scorrere i bit verso destra e quando il bit che esce è 0 significa che l'operazione è stata concessa, poiché quello è l'istante in cui è stato calcolato che non c'erano conflitti se partito in quell'istante. Questi registri sono la memoria: ricordano la storia passata delle operazioni.

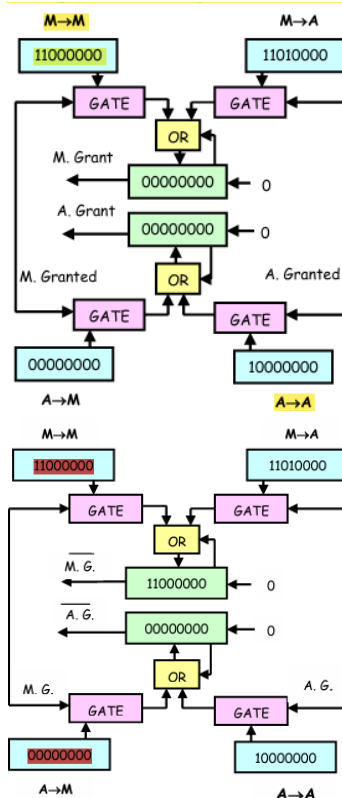
In ingresso nello shift c'è sempre uno 0.

Le GATE sono semplicemente dei tornelli che lasciano passare i vettori delle collisioni solo se l'istruzione è appena stata avviata in modo da ricalcolare lo shift register.

Le porte OR fondono i vecchi divieti (che si trovano già nello shift register) con i divieti della nuova istruzione. Questo è fondamentale perché devo essere in grado di poter gestire anche una sequenza di più operazioni.

Se facessimo la AND sbaglieremmo perché la OR consente di imporre la logica che se la vecchia istruzione mi impone un divieto (1) **OPPURE** la nuova istruzione mi impone un divieto (1), allora io devo mantenere il divieto (1).

1.2 Esempio:



all'inizio abbiamo questa situazione con i vettori dei conflitti già caricati e gli shift register sono tutti a 0.

La prima operazione è una MUL.

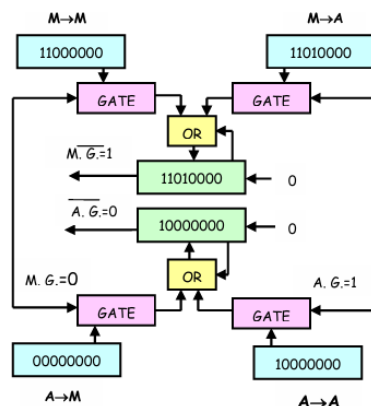
Poiché sulla prima operazione non abbiamo problemi si alza Mul granted. M.garanted è collegato ai due vettori in cui M è la prima istruzione. Quindi si fa la OR tra M->M e lo shift di sopra e poi la OR tra A-> e lo shift di sotto.

gli shift sono correttamente caricati.

Avanza il clock di uno e succede che gli shift shiftano a sinistra. Entra uno 0 a destra e arriva la seconda operazione che è una ADD. Siccome è uscito uno 0 dallo shift di sotto il segnale ADD grant consente di far partire la ADD. Il comportamento si trova con quello che sta succedendo?

Si perché A->M significa prima ho una mul poi una add. Queste due non generano conflitto neanche al primo clock e infatti add grant ci ha fatto passare.

Nel mentre quindi si alza il signal ADD granted perché la ADD è passata e quindi è entrata nella pipe, in modo che fa sbloccare la OR tra i registri M->A e A->A perché ora l'istruzione ultima è una ADD e devo capire quella dopo se ha conflitto.



Il segnale Add Granted permette di caricare i vettori M→A (110100) e A→A (100000) nelle OR, che calcolano i nuovi valori dei registri, ottenendo rispettivamente:

- 1) 110100
- 2) 100000

Arriva il secondo clock.

I registri shiftano quindi diventano

10100000

00000000

Dagli shift sono usciti degli uno quindi sia se è arrivata una nuova add che una nuova mul sono bloccate.

Ora potrei avere due strade:

- 1) Al secondo clock non arriva nessuna operazione → gli shift rimangono uguali
- 2) Al secondo clock arriva un'operazione ma essendo che sono usciti degli 1 non posso fare entrare nella pipe né la MUL né la ADD quindi i segnali granted rimangono bassi.

Nel caso in cui non arrivasse nulla al 2 clock vediamo cosa succede al terzo clock.

Al terzo clock abbiamo:

1 | 01000000

0 | 00000000

Quindi se al terzo clock arriva una ADD la posso fare perché è uscito 0, quindi entra nella pipe, si alza ADD granted e aggiorno gli shift, se arriva una mul non la posso fare. E così via.

Quindi quella OR ci prepara a capire la prossima istruzione che arriva se la posso fare o meno sulla base dell'istruzione precedente e di quelle ancora prima. Invece il bit che esce mi dice se l'istruzione di ora la posso fare oppure no. La OR avviene solo nel momento in cui un'operazione entra nella pipe perché così posso calcolare i divieti con l'operazione successiva.

Se avessi lo shift con tutti 1 quindi 11111111

Scatta il clock e arriva una MUL 1 | 11111110 la MUL è bloccata

Un altro clock 1 | 11111100

1 | 11111000

1 | 11110000

1 | 11100000

1 | 11000000

1 | 10000000

1 | 00000000

0 | 00000000 a questo ciclo di clock il segnale Mul grant mi dà il permesso di caricare una MUL. La MUL stava aspettando e allora si alza anche MUL granted ed entra nella pipe, aggiornando lo shift

11000000 in modo che ora so che se arriva un'altra MUL non la posso fare

1.3 Salti nelle architetture superscalari

In una pipeline singola e semplice, se il processore "tira a indovinare" la strada sbagliata, deve bloccare la pipe, buttare via le 2 o 3 istruzioni che aveva già prelevato (operazione chiamata *flush* o svuotamento) e ricaricare quelle giuste. È un danno, ma contenuto.

In un'architettura **superscalare**, il danno è devastante:

- A causa di un salto, "bisogna arrestare non solo una pipe, ma più pipe che operano in contemporanea, e gestire di conseguenza il loro riavvio".
- Se hai 4 pipe parallele e ti accorgi dopo 3 cicli di clock di aver sbagliato strada, non devi buttare via 3 istruzioni, ma ne devi buttare via **12** (4 pipe x 3 cicli) che stavano allegramente calcolando dati inutili!

L'impatto sul tempo sprecato e sull'energia consumata è enorme, specialmente considerando che, i salti sono frequentissimi: rappresentano circa il 25% delle istruzioni in un programma tipico. In pratica, un'istruzione su quattro rischia di far deragliare tutte le pipe.

Per evitare che il processore si blocchi in continuazione possiamo dedicare una specifica pipe alla predizione dei salti. Ovviamente la predizione deve essere fatta il prima possibile così da ridurre al minimo gli inconvenienti all'interno delle altre pipe.

2. Memoria

L'obiettivo ideale è costruire una memoria che sia contemporaneamente **veloce e molto capiente**. Poiché fisicamente ed economicamente non è possibile, si ricorre a una gerarchia di memorie: si affiancano memorie piccole e veloci a memorie più lente ma molto capaci. Avendo architetture pipelined o architetture superscalari, la frequenza con cui riusciamo a lavorare è estremamente elevata, per cui se in un colpo di clock non riusciamo ad avere il nuovo dato, la nuova istruzione, perdiamo i benefici della velocità interna al processore.

La memoria principale viene realizzata normalmente utilizzando componenti **DRAM** (Dynamic Random Access Memory, ovvero memoria dinamica ad accesso casuale).

Memorie più veloci (tipicamente usate per le Memorie Cache) sono composte da **SRAM** (Static Random Access Memory). Il costo per bit delle DRAM è più basso di quello delle SRAM; d'altra parte, le prime sono molto più lente delle seconde. La differenza di prezzo dipende dal fatto che le memorie DRAM utilizzano meno transistori per ogni bit da memorizzare: consentono quindi di raggiungere capacità superiori a parità di area di silicio.

Da qui nascono due tecnologie principali:

- **Memoria Virtuale**: La gestione è mista hardware/software. Se l'informazione è presente, interviene l'hardware. Se non c'è (Page Fault), si deve attivare un driver (software) che, tramite DMA, va a leggere la pagina mancante dal disco. Non si ha la certezza che il dato cercato si trovi in RAM.
- **Memoria Cache**: La gestione (sia in caso di dato trovato che non trovato) è **completamente in hardware**. Se il dato non è in cache, l'hardware si occupa di andarlo a cercare in RAM. L'efficacia della cache si basa sul concetto probabilistico di **località**: è fondamentale scrivere programmi che abbiano un alto grado di località spaziale e temporale. La cache è una memoria estremamente veloce ma di dimensioni molto ridotte (nell'ordine dei MB), che si interpone tra il processore e la memoria RAM (nell'ordine dei GB). Poiché lo spazio di indirizzamento del processore è enormemente superiore alla capacità della cache (sei ordini di grandezza di differenza), è fisicamente impossibile caricare tutti i dati al suo interno.

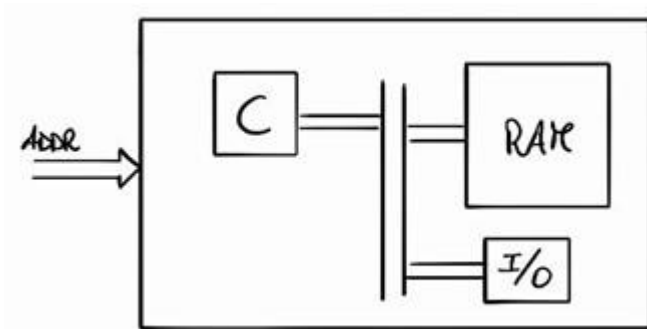
A livello architetturale, il processore comunica con la memoria attraverso un Bus di Sistema, che è concettualmente suddiviso in tre gruppi di linee distinte:

- **Bus Indirizzi (Address Bus)** – Costituito da k linee. Determina lo spazio di indirizzamento massimo del sistema, pari a 2^k locazioni;
- **Bus Dati (Data Bus)** – Costituito da n linee. Determina la quantità massima di bit (parola di trasferimento) che possono viaggiare in parallelo in un singolo ciclo;
- **Bus di Controllo (Control Bus)** – Trasporta i segnali (es. Read, Write, Memory Function Completed) necessari per implementare il protocollo di sincronizzazione tra processore e memoria, essenziale poiché i due componenti lavorano a velocità differenti.

Il processo fondamentale di interfacciamento avviene tramite una coppia di registri dedicati all'interno della CPU: **MAR (Memory Address Register)** – Il processore vi inserisce l'indirizzo della locazione di memoria a cui vuole accedere. Questo valore viaggia sul Bus Indirizzi per decodificare e selezionare la cella

specifica; **MDR (Memory Data Register)** – Funge da buffer. Durante una lettura, i bit estratti dalla memoria viaggiano sul Bus Dati e vengono depositati nel MDR, rendendoli accessibili al data path del processore. Durante una scrittura, avviene il processo inverso.

2.1 Trattazione senza memoria virtuale



Il processore lavora con gli **indirizzi fisici** e richiede un dato senza sapere a priori se questo si trovi in cache o meno.

- La prima richiesta viene sempre fatta alla cache.
- Se c'è un *Cache Hit*, il dato viene restituito.
- Se c'è un *Cache Miss*, l'hardware va a recuperare il dato in RAM.

Normalmente i sistemi adottano un **doppio livello di cache (L1, L2)**:

- **Cache L1**: Più piccola, integrata direttamente nel SoC (System on Chip) del processore.
- **Cache L2**: Più grande e posta all'esterno del core principale.
- **Flusso**: Il processore cerca prima in L1, se non trova cerca in L2, e solo in caso di doppio miss va in RAM.
- Tra le due cache esiste un bus dedicato. Poiché non ci sono conflitti con i dispositivi di I/O su questo bus, tra L1 e L2 è auspicabile usare la tecnica **Write-Through**.

Questa complessa organizzazione permette di creare **un'illusione ottica per il processore**:

- **Illusione di Velocità** – Il processore ha quasi sempre l'istruzione o il dato pronti in un ciclo di clock (tempi di trasferimento nanosecondi), "credendo" che tutta la memoria sia veloce come il livello superiore;
- **Illusione di Spazio** – Il processore percepisce una memoria apparentemente immensa, in grado di sfruttare tutto il suo spazio di indirizzamento (es. con registri a 64 bit può indirizzare teoricamente fino a 264 locazioni), anche se fisicamente solo una minima parte di questi dati risiede nelle memorie veloci in quel momento.

Anche se una gerarchia di memoria è in genere composta da più livelli, le informazioni vengono di volta in volta copiate **solo tra due livelli adiacenti**.

Località temporale: quando si fa riferimento a un elemento di memoria, c'è la tendenza a far riferimento allo stesso elemento entro breve (un tipico esempio è costituito dal riutilizzo di istruzioni e dati contenuti nei cicli);

Località spaziale: quando si fa riferimento a un elemento di memoria, c'è la tendenza a far riferimento entro breve tempo ad altri elementi che hanno indirizzo vicino a quello dell'elemento corrente (tipicamente, eseguita un'istruzione si tende ad eseguire quella immediatamente successiva; quando si

accede a dati organizzati in vettori o matrici, l'accesso a un dato è seguito dall'accesso al dato immediatamente successivo, etc.).

L'utilizzo della cache ci pone davanti 3 problemi fondamentali:

- 1) Come detto in precedenza la RAM è molto più grande rispetto alla cache e quindi come possiamo mappare i tantissimi indirizzi della RAM nei pochi indirizzi della cache?
- 2) La cache si riempie in fretta. Quale blocco posso sostituire per fare spazio ad un altro blocco che deve essere caricato nella cache?
- 3) **Coerenza**: quando il processore modifica un dato che si trova nella cache, deve essere ricopiato in RAM poiché altrimenti si creerebbe un disallineamento con il dato in memoria.

Esistono due tecniche principali per gestire la coerenza:

- **Write-Through (Scrittura immediata)**: Ogni volta che si scrive in cache, il dato viene copiato istantaneamente anche in RAM.
 - **Svantaggi**: Crea molti conflitti sul bus di sistema, specialmente con i dispositivi di I/O (es. se c'è un controller DMA attivo).
 - **Vantaggi**: In caso di guasto della cache, i dati in RAM sono sempre aggiornati e sicuri. Se avviene un fallimento in lettura cioè un cache miss non è un grandissimo problema perché non richiede la scrittura al livello superiore.
- **Write-Back (Scrittura differita)**: Si scrive solo in cache. Il dato viene copiato in RAM solo quando il blocco deve essere rimosso dalla cache per fare spazio a nuovi dati.
 - **Vantaggi**: Richiede molte meno scritture in memoria rispetto al Write-Through, alleggerendo il bus. Le parole possono essere scritte dalla CPU alla frequenza a cui la cache è in grado di accettarle, invece di preoccuparsi della memoria. Se abbiamo un fallimento di lettura la situazione è più complessa perché se il blocco da sostituire ha il bit dirty =1 deve essere scritto in memoria.
 - **Nota**: Questa differenza di politiche riguarda solo la *Cache Dati*. Nella *Cache Istruzioni* (che viene solo letta e mai scritta), queste tecniche non si applicano.

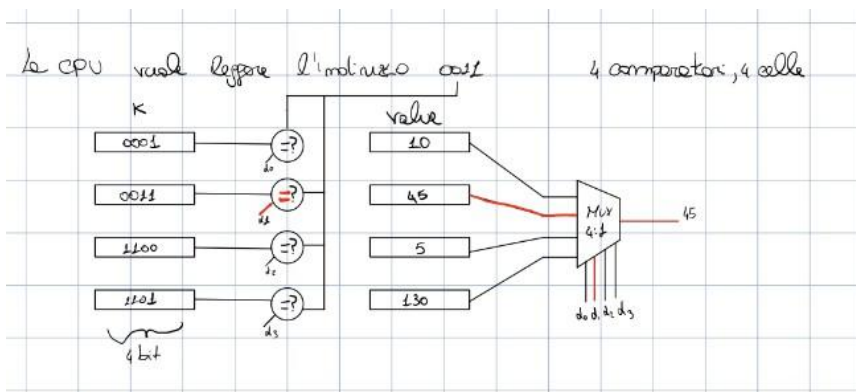
Come possiamo attuare queste tecniche ai diversi livelli di cache?

- Tra la cache che si trova nel SoC del processore e la cache di livello immediatamente superiore si può utilizzare la politica Write-Through poiché utilizzano un bus dedicato che è molto rapido. Quando C1 modifica un dato, questo viene aggiornato istantaneamente in C2. Quindi quando C1 è piena e deve sostituire un blocco può scartarlo e sovrascriverlo.
- Tra la cache non primaria e la RAM si usa la politica Write-Back poiché sono collegati al bus principale che inoltre è condiviso anche con le altre periferiche di I/O.

2.2 Tecniche di gestione

A differenza di un normale decoder in RAM (che seleziona sempre una linea esistente), in cache non si ha la certezza che l'indirizzo esista. L'indirizzo a 32 bit funziona come una **chiave di ricerca**.

2.2.1 Memoria Totalmente Associativa (Una chiave = Un elemento):



la cache è composta k comparatori che indicano il grado di associatività. Il comparatore compare l'intero indirizzo (la chiave) con gli indirizzi memorizzati nelle chiavi della cache.

Abbiamo un match 1:1 tra chiavi e valori, quindi il grado di associatività indica anche la

dimensione della cache.

Se ho 4 comparatori ho quindi 4 locazioni, se ogni locazione è di un byte allora la dimensione della cache è di 4 byte. Per aumentare la capacità della cache devo necessariamente aumentare il numero di comparatori. Se volessi una cache di 1 KB = 1024 byte avrei bisogno di 1024 comparatori, il che significa un'occupazione di silicio enorme, sarebbe una situazione ingestibile. Inoltre per avere una cache di 1 KB, devo memorizzare anche 1024 chiavi. Se gli indirizzi sono a 32 bit, cioè una RAM di 4 GB, significa che devo memorizzare per la cache 1024 indirizzi da 32 bit cioè $1024 \times 32 = 32768 \text{ bit} = 4096 \text{ byte} = 4 \text{ KB}$.

Quindi per un solo KB di dati devo memorizzare 4 KB di chiavi. Altamente inefficiente.

Quando c'è un miss e la cache è piena, bisogna decidere quale blocco espellere. A questo scopo, ogni cella della cache contiene dei bit di gestione (overhead logico):

- **Dirty Bit (Bit di modifica):** Fondamentale nella tecnica Write-Back. Messo a '1' indica che il blocco è stato modificato rispetto alla RAM. Se il bit è a 1, prima di espellere il blocco, l'hardware deve salvarlo in RAM per non perdere i dati. Se è a 0 (solo letto), può essere sovrascritto direttamente.
- **Bit di Storia / Contatore (es. 2 bit):** Servono a gestire le politiche di sostituzione basate sull'utilizzo (come l'algoritmo LRU - Least Recently Used).
 - *Esempio:* Ogni volta che non si accede a una cella, il contatore si incrementa. Valore 0 = usata di recente; Valore 3 = non acceduta da molto tempo (candidata ideale per l'espulsione).
- **Bit di validità:** serve per capire se la cache contiene dati validi ad esempio quando il processore viene acceso la cache è vuota e le informazioni nelle etichette non hanno nessun significato. Il bit di validità è indipendente dall'architettura scelta, è presente in tutte le soluzioni.

Se si sceglie una politica di sostituzione *casuale (Random)*, questi bit di storia non sono necessari.

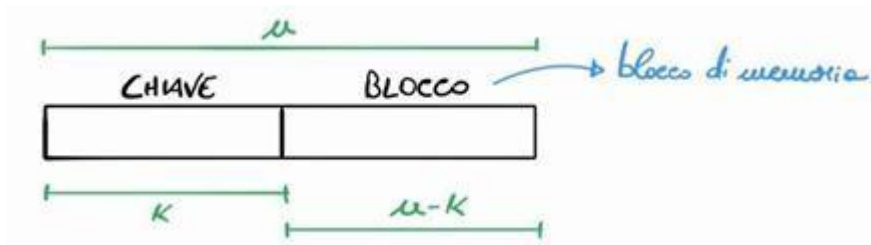
La dimensione della cache è pari al grado di associazione che coincide con il numero di comparatori. Inoltre non sfruttiamo per niente il concetto di località spaziale perché possiamo avere nelle chiavi degli indirizzi totalmente sconnessi tra loro.

2.2.2 Organizzazione a Blocchi (Divisione dell'indirizzo):

Vorremmo in qualche modo risolvere l'inefficienza dello spazio e sfruttare la località spaziale, oltre che ridurre l'elevato numero di comparatori utilizzato nella cache totalmente associativa. In questo modo possiamo avere non più una singola locazione, ma un blocco di indirizzi contigui.

L'indirizzo viene diviso in 2 parti:

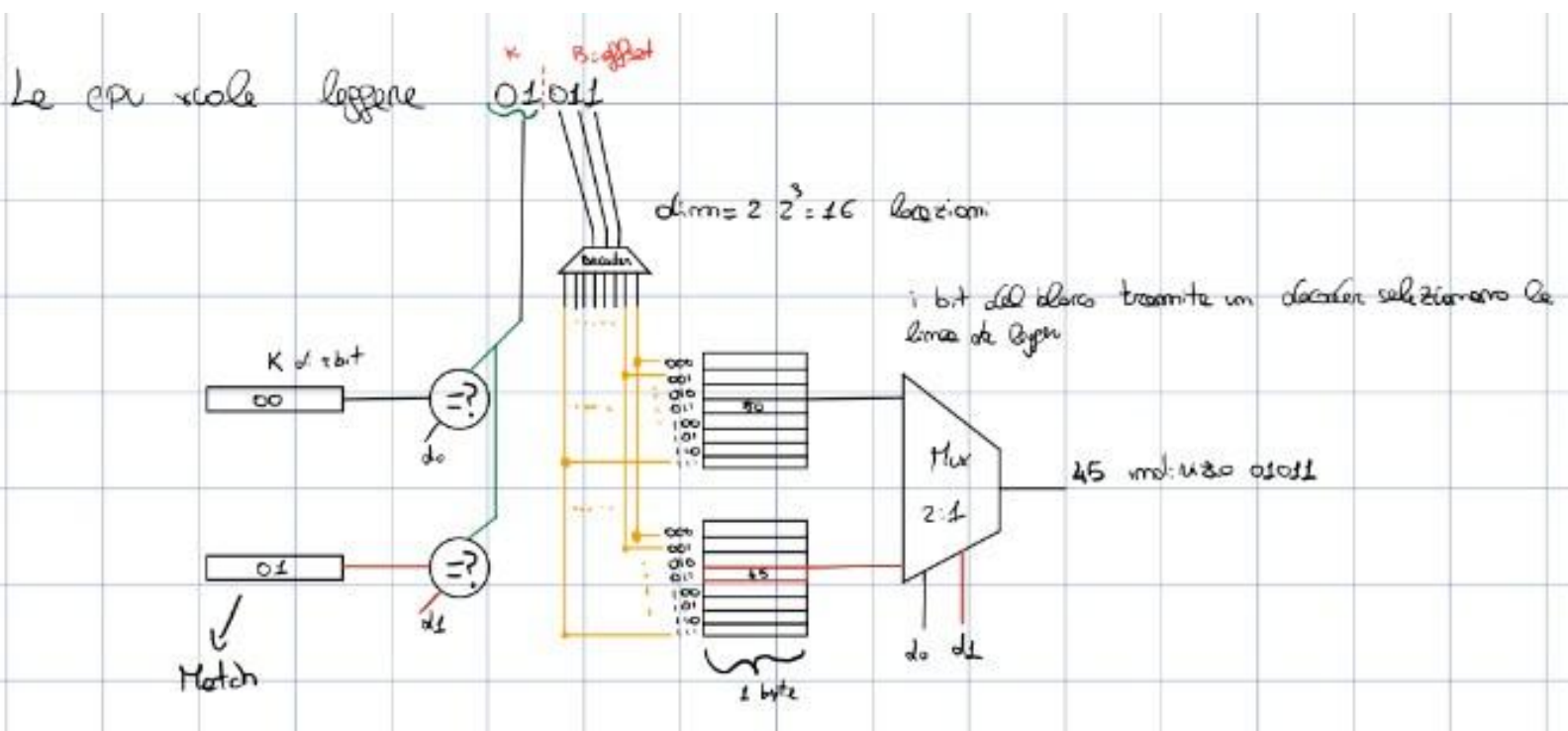
- Chiave(tag): identifica l'intero blocco
- Blocco (offset): indica la posizione esatta della singola parola desiderata all'interno del blocco caricato.



In questo caso per ogni chiave abbiamo un blocco di più locazioni, questo ci consente di ridurre notevolmente i comparatori a parità di locazioni di cache disponibili.

Il numero di comparatori è il grado di associatività in cui all'interno scriviamo le chiavi. Una chiave non è tutto l'indirizzo ma solo una parte cioè i k bit dell'indirizzo.

I restanti $u-k$ bit indicano la grandezza del blocco ovvero 2^{u-k}



I bit di modifica e di storia vengono associati all'intero blocco e non alla singola informazione.

Consideriamo il caso in cui il Tag è di 2 bit su 5 quindi abbiamo 3 bit di blocco. Ogni blocco è quindi di 8 locazioni. Per ogni TAG abbiamo a disposizione tutto il blocco da 000 a 111 quindi poi grazie a quei 3 bit riusciamo a selezionare la linea corrispondente.

Abbiamo bisogno di un decoder per selezionare la singola riga di ogni blocco e poi il match con un comparatore porta alla selezione del byte del blocco corretto.

In questo esempio il grado di associatività è pari a 2. Il grado di associatività non è legato al numero di bit della chiave.

La dimensione della cache è quindi $= \text{grado di associatività} \times \text{dim_blocco} = 2 \times 8 = 16$ locazioni ognuna da 1 byte quindi 16 byte. Poi per memorizzare le chiavi abbiamo $2 \times 2 \text{ bit} = 4 \text{ bit}$

Prendiamo come esempio l'indirizzo 24

In binario 01011

La chiave è 10 che matcha con il secondo. Sono sicuro che se matcha con il secondo comparatore e fa alzare la linea alfa1 del MUX. A questo punto il decoder prende 011 e fa alzare la quarta linea di ogni blocco. Il MUX fa uscire poi la linea corretta.

Questa architettura favorisce la località spaziale perché quando viene caricato un dato nella cache in realtà non carichiamo soltanto lui, ma tutto il blocco e quindi se ad esempio abbiamo un vettore con valori tutti vicini li abbiamo già in cache.

Prendiamo un caso in cui gli indirizzi sono a 32 bit, quindi la RAM è di 2^{32} locazioni. Se ogni locazione è di 1 byte allora la RAM è di 4 GB.

Vogliamo una cache di 1KB = 1024 byte

Dei 32 bit possiamo ad esempio avere che i primi 24 bit identificano la chiave e gli 8 bit finali il blocco.

Ogni blocco è di 2^8 locazioni ovvero 256 locazioni di un byte quindi 256 byte. Per avere una cache di 1024 byte dobbiamo usare $1024/256 = 4$ comparatori

Quindi memorizziamo 4 chiavi nella cache di 24 bit, 4 comparatori e la cache ha dimensione di 1024 byte.

Per 1024 byte devo memorizzare $4 \times 24 \text{ bit} = 12 \text{ byte}$.

Il vantaggio di avere blocchi più grandi è che devo usare meno spazio per le chiavi, ma ciò significa che per ogni dato che metto in cache mi porto dietro più byte che magari potranno anche non essere mai usati. Comunque è molto più conveniente rispetto a prima perché ho 4 comparatori soltanto in confronto ai 1024 di prima.

2.2.3 Cache set-associativa a settori

Se il programma in esecuzione è molto locale, allora avere blocchi più grandi è sicuramente conveniente, mentre se il programma è poco locale, la cosa migliore è avere molti blocchi ma piccoli.

Notiamo inoltre che maggiore è la dimensione del blocco e maggiore è la quantità di dati da trasferire quando si presenta un cache miss.

Avere tanti blocchi però comporta avere tanti comparatori e avere tante chiavi comporta lo spreco nel memorizzarle.

Per evitare di utilizzare blocchi giganti e ridurre il numero di comparatori è possibile dividere l'indirizzo di memoria in 3 parti:

- Chiave del blocco
- Settore (index)
- Blocco (offset)



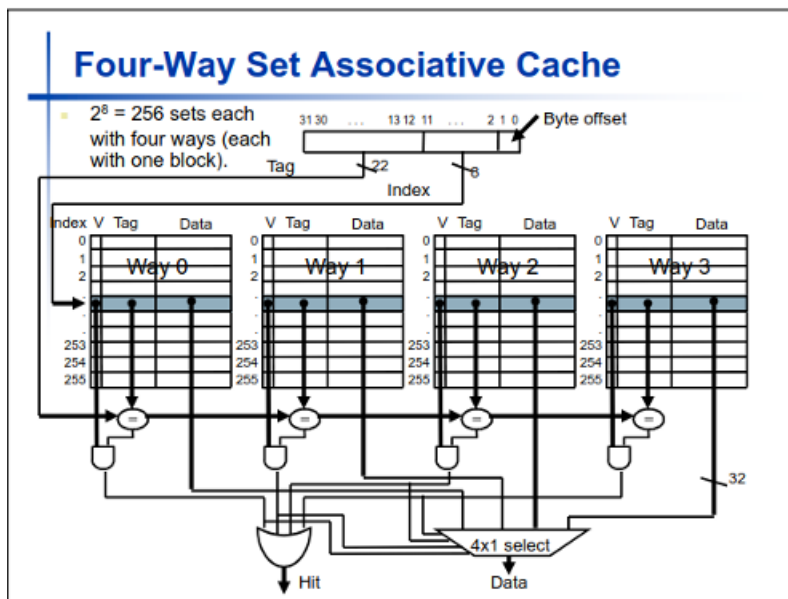
il settore funziona da indice per un indirizzamento diretto.

Durante la decodifica dell'indirizzo, l'hardware utilizza quei 2 bit per selezionare un singolo set della cache, restringendo lo spazio di ricerca.

Tale architettura è una via di mezzo tra cache diretta e fully associative.

La cache è divisa in molteplici set ognuno contenente tanti blocchi. Un dato può trovarsi in qualsiasi blocco del set, ma la ricerca avviene soltanto nel set giusto individuato dai bit di settore.

- 1) Selezione del set: utilizzando i bit dell'indirizzo destinati all'indice del set, il decoder seleziona l'insieme corretto di blocchi della cache. Solo i blocchi di questo set saranno coinvolti dalla ricerca.
- 2) All'interno di questo insieme, i tag dei blocchi vengono confrontati in parallelo con il tag dell'indirizzo. Questo confronto determina se il blocco cercato è presente o meno. Le uscite dei comparatori determinano tramite il MUX quale dato deve essere preso tra quelli selezionati nei blocchi grazie ai bit di offset.



Nella full associative confrontiamo con tutti i blocchi mentre in questa solo con i blocchi selezionati nel set

Con 2 bit di settore significa avere 4 settori e con 10 bit di blocco significa che ogni blocco ha 2^{10} locazioni.

Set associative ad n vie dove le n vie rappresentano il numero di blocchi che possiamo avere all'interno di un insieme.

Significa quindi che per ogni chiave ho $num_settori \times num_blocchi = 4 \times 2^{10}$

In totale la dimensione della cache è quindi $= dim.blocco \times g.associatività \times n.settori$

Questa implementazione abbassa drasticamente la complessità circuitale, lo spazio occupato sul silicio e il ritardo di propagazione delle porte logiche, rappresentando il compromesso ingegneristico ottimale tra l'efficienza di un mapping completamente associativo e l'economicità di un indirizzamento diretto. Il problema principale è quello legato ai comparatori e noi vogliamo avere una cache sempre più grande ma con pochi comparatori e con i 3 modelli abbiamo visto come con gli stessi comparatori abbiamo aumentato il numero di locazioni.

Precisiamo che nel settore non c'è scritto niente ma individua direttamente il blocco che vogliamo prendere, quindi se $s=00$ allora è il blocco 0, se 01 è il blocco 1 e così via.

Quando c'è un cache miss e dobbiamo prendere in RAM il byte per metterlo in cache, noi portiamo tutto il settore, ma soltanto il blocco di quel byte quindi 2^{10} byte.

A questo punto verrebbe da chiederci ma gli altri blocchi di quel settore dove sono?

Utilizziamo un bit di validità. All'inizio dopo un cache miss e dopo aver deciso quale chiave sacrificare, vengono impostati tutti e 4 i bit di validità a 0 (i bit del settore).

In memoria viene prelevato soltanto il blocco richiesto (2^{10} byte) non tutti e 4, lo mette nel blocco giusto guidato dai bit del selettore e mette a 1 solo il bit di validità di quel singolo blocco.

A questo punto quindi può capitare un miss di blocco:

La CPU chiede un dato. Il Tag fa "Hit" (la chiave è nei comparatori!), quindi il settore è quello giusto. Però, i 2 bit di settore puntano, ad esempio, al Blocco 3. L'hardware guarda il Valid Bit del Blocco 3 e vede che è a 0 (il blocco non è mai stato caricato). Cosa fa l'hardware?

1. La chiave c'è già, quindi non tocca nulla a livello di Tag.
2. Va in Memoria Principale e preleva **SOLO il Blocco 3** (1024 byte).
3. Lo copia nello spazio del Blocco 3 in cache e mette il suo Valid Bit a 1.

Il motivo per cui si fa così è che potremmo creare un collo di bottiglia verso la memoria perché se ad ogni cache miss dobbiamo caricare tutto il settore, potremmo intasare il bus per trasferire un sacco di dati che magari non useremo mai.

Caricare solo il blocco richiesto permette di avere il meglio dei due mondi:

- **Risparmi hardware:** Meno Tag RAM e meno comparatori (perché usi un Tag per raggruppare più blocchi).
- **Risparmi banda di memoria:** Trasferisci dal bus solo il singolo blocco da 1 KB di cui hai realmente bisogno in quel momento.

L'unico problema di questa architettura è che abbiamo una bassa efficienza di dati, cioè se carichiamo solo un blocco stiamo perdendo il 75% di spazio di quel settore. Ci accontentiamo di questa architettura perché offre una località spaziale veramente eccellente.